

Final Project Report

FPGA Audio Mixer

Alexi Carlone and Iris Kaucher

Project Description

Our goal for this project was to take in 8 wave files as audio stems and mix them together based on user input, outputting one result that has been altered algorithmically to be at the correct levels. We use the Pynq framework running Jupyter Notebook to load audio files in arrays and produce the final audio output and Xilinx HLS tools to mix together audio samples and run other audio processing algorithms on top to make the audio sound better.

Code Description and Inputs

For this project we are using stems from Jamie Paige's song "ringtone", which we have obtained under a Creative Commons BY-NC-SA license. These stems have been modified using Audacity to a 16-bit mono format. These stems all were loaded onto the Jupyter Notebook of the AUP-ZU3 board to be loaded by the software.

Software Description

The software running on the PYNQ Jupyter Notebook loads all the wave files into numpy arrays of integers. It ensures that the audio it's receiving is 16-bit so there's no issues with processing later down the line. Then, these individual arrays of samples are packed together into a single array. Because the AXI DMA requires 32-bit integer arrays, we pack the 8 16-bit integers into sequences of 4 32-bit integers. In addition, the stem files may be of slightly different lengths, so we pad out the arrays to make them the same length. These files are then transferred to the DMA in chunks of 8188 integers and then received because transferring the entirety of all 8 files exceeds the buffer size of the DMA. After receiving the final mixed buffer as a sequence of 32 integers, the integers are processed into floats, mapped to a -1.0 to 1.0 range, and then outputted as an audio file in Jupyter Notebook using the IPython Audio module.

Hardware Description

The hardware has three main algorithms: attenuation, compression, and smoothing. The first of these works to scale the mixed audio's output volume so that as the number of input stems changes the output volume remains similar. This is necessary as the stems are mixed through simple addition, without this step the volume would scale directly with the number of stems. The algorithm uses the formula shown below.

$$mix = \sum(stems) / \sqrt{(\# \text{ of inputs})}$$

The compression algorithm works to reduce the magnitude of the mix when it exceeds a set threshold, in our case this is set to -6dB. When the magnitude of our mix exceeds this threshold by 3dB or more we apply the following equation.

$$outputDB = (mixDB - T) / R + T$$

Here T is the threshold mentioned before and R is a compression ratio. We chose to use a ratio of 4 as it is a nice balance between providing noticeable compression without going overboard. Additionally, this is done in decibels rather than linearly unlike the attenuation algorithm. The mix is then subtracted from the output here to get a gain, which we then convert back from decibels to so it can be easily multiplied with our mix from the attenuation algorithm. When the magnitude of our mix sits within 3dB of our threshold we apply a soft knee compression algorithm. This smooths the compression so its ratio scales from 0 to 4 as approach and pass the threshold removing any potentially jarring changes in volume caused by sudden compression as the mix gets louder. The soft knee equation can be seen below.

$$(1/R - 1) \times (mixDB - T + KneeWidth/2)^2 / (2 \times KneeWidth)$$

The compression was calculated for a wide range of volumes and put into a lookup table in our implementation so that these calculations could be avoided at runtime. We used a LUT of width 512 for this and avoided including most of the uncompressed values as they all have a gain of 1. This allowed us to still have a large number of gains within our compression range without using too much LUT space on our FPGA. One effect of this is that our gain doesn't form a smooth curve, to account for this we applied a final smoothing algorithm. To do this we take the target gain from our LUT and subtract our current gain from it to get the difference. We then multiply this difference by either an attack or release coefficient depending on whether our gain is decreasing or increasing. These coefficients are as follows.

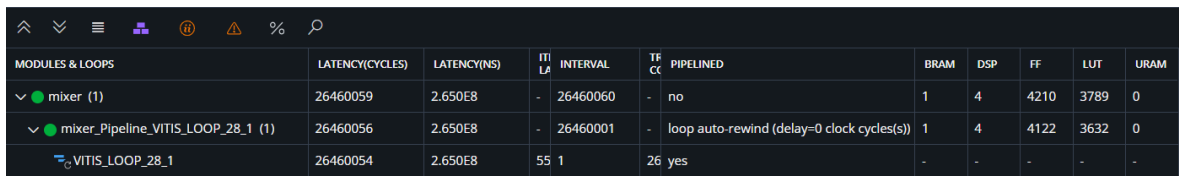
$$\text{Attack_coeff} = 0.00226 \text{ (~10ms at 44.1kHz)}$$

$$\text{Release_coeff} = 0.000151 \text{ (~150ms at 44.1kHz)}$$

These result in a much smoother transition as the gain changes. Finally we have a simple brick-wall limiter that caps the magnitude of our output at 0dB (an amplitude of 32,767 in our code) to avoid any uncomfortably loud outputs. We expect this limiter to take effect rarely if ever thanks to the compressor but it exists as a failsafe just in case.

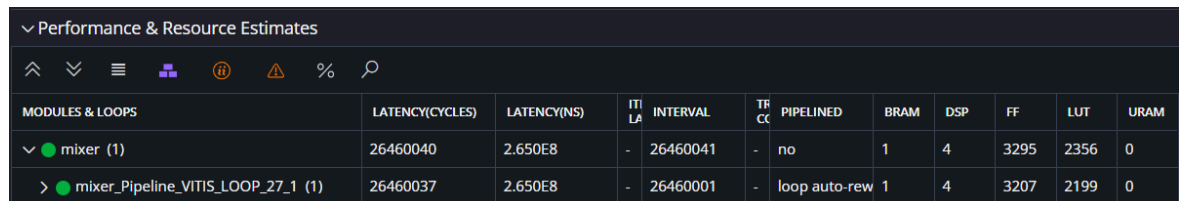
Design Space Analysis

Because the use case of our project involves a lot of known ranges of values, particularly around volume and number of input stems, we were able to do most of our math at design time and put the results in LUTs to reduce the number of calculations at runtime. This meant that we did not have much room for design space analysis at runtime. The key area where we were able to make changes was surrounding the attenuation algorithm. We initially had the full algorithm being calculated at runtime but found this to be quite space inefficient. The synthesis output can be seen below.



MODULES & LOOPS	LATENCY(CYCLES)	LATENCY(NS)	ITI LA	INTERVAL	TR CC	PIPELINED	BRAM	DSP	FF	LUT	URAM
mixer (1)	26460059	2.650E8	-	26460060	-	no	1	4	4210	3789	0
mixer_Pipeline_VITIS_LOOP_28_1 (1)	26460056	2.650E8	-	26460001	-	loop auto-rewind (delay=0 clock cycles(s))	1	4	4122	3632	0
VITIS_LOOP_28_1	26460054	2.650E8	55	1	26	yes	-	-	-	-	-

As you can see, the area usage is quite significant. We then decided to put our $\sqrt{\# \text{ of inputs}}$ in a LUT and found this greatly improved area usage. The results can be seen below.



MODULES & LOOPS	LATENCY(CYCLES)	LATENCY(NS)	ITI LA	INTERVAL	TR CC	PIPELINED	BRAM	DSP	FF	LUT	URAM
mixer (1)	26460040	2.650E8	-	26460041	-	no	1	4	3295	2356	0
mixer_Pipeline_VITIS_LOOP_27_1 (1)	26460037	2.650E8	-	26460001	-	loop auto-rew	1	4	3207	2199	0

This was a big improvement on area usage and minor improvement in latency but still not our best solution. We found that by storing the reciprocal square root in our LUT and doing multiplication at runtime rather than division was even more efficient.

Performance & Resource Estimates											
⌵ ⌶ ☰ 🧩 ⌚ ⚠ % 🔍											
MODULES & LOOPS	LATENCY(CYCLES)	LATENCY(NS)	ITI LA	INTERVAL	TR CC	PIPELINED	BRAM	DSP	FF	LUT	URAM
▼ ● mixer (1)	26460010	2.650E8	-	26460011	-	no	1	5	448	950	0
> ● mixer_Pipeline_VITIS_LOOP_27_1 (1)	26460007	2.650E8	-	26460001	-	loop auto-rew	1	5	360	793	0

Once again the effect this had on latency was a very minor improvement but the impact on area usage was significant. Because we found this to be true before ever implementing the compression algorithm, we never attempted to do compression at runtime and opted to use a LUT from the start. As a result we never performed design space analysis on the other algorithms. We also tried doing a complete partition of both LUTs, we found however that this made no meaningful impact to latency and increased area usage so we opted not to move forward with it. We also attempted to implement pipelining but couldn't seem to make it work. Due to time constraints and that our algorithm already runs roughly 700 times faster than the speed of our tracks we opted not to troubleshoot this.

Testing

For testing we wrote a testbench that tests our three algorithms across four tests. The first of these is a simple passthrough test, providing one input sine wave well below threshold to see if the output matches what we expect. The second test tests our attenuation algorithm by providing 3 sine waves as inputs with amplitudes well below our threshold to see that they mix as expected. The third test ensures our brick-wall limiter works by providing eight inputs at amplitudes of 30,000 and checking that the output never exceeds 32,767. Our fourth and final test is more visual, taking in one stem above our threshold (at an amplitude of 25,000) then cutting it to 0 half way through. We expect to see the output magnitude decrease over time as the compressor and smoothing take effect. As you can see below, all of our tests passed as expected.

```

Test 1 (passthrough): PASS (peak=4000, errors=0)
Test 2 (sqrt mixing): PASS (peak=6000, errors=0)
Test 3 (limiter ceiling): PASS (peak=32767, errors=0)
Test 4 (envelope) â€" output samples at key points:
n= 0 out= 22656
n= 10 out= 22558
n= 50 out= 22167
n= 200 out= 20996
n= 1000 out= 18652
n= 2200 out= 18261
n= 2205 out= 0
n= 2250 out= 0
n= 2500 out= 0
n= 3000 out= 0
n= 4409 out= 0
(expect: gain pulls down over first ~500 samples,
recovers over ~5000 samples after burst ends at n=2205)
=====
ALL TESTS PASSED (0 failures)
=====

```

Lessons Learned

Originally, we wanted to have 8 streams for each audio file running into the DMA, but we found it was hard to sync up 8 streams that need to be simultaneously added together. We learned we could solve this by packing all the files into a singular, very wide stream, though it did mean we had to make the resolution of the music slightly lower and take the files from 24-bit to 16-bit.

We also found that transferring entire minute-long audio files at once causes issues for the DMA, as the file size is too big and overflows the buffer. We solved this by chunking up the data into smaller segments, though it does cause issues because we need to constantly start the mixer block up and make many DMA transfers which could worsen the time to do processing by a lot.

Finally, we found that we couldn't change what stems are mixed together with the switches while the mixer was running because it calculates it so fast. If we were to want to do that, we would need to slow the Vitis HLS mixer block way down.

Future Work

With more time, we would like to get output to the built-in audio codec working. We attempted to do this for a very long time and got the codec constrained and communicated with, but the I2S transmitter wouldn't consume the stream from the mixer for whatever reason and the whole thing would stall. Getting this working would take a lot of time and debugging using ILAs on the FPGA.

We would also want to make the switches sync with audio output. This is only really viable if using the audio codec, but we would want to make it so changing the switches changes the final mix while it is mixing. This would probably require slowing the mixer block down by a lot to sync it up with the audio output.

In addition, we would like to add more audio algorithms to the mixer. We would want to add an EQ so we can filter out low noise that gets introduced when mixing files or maybe sidechaining to make certain stems such as the vocals stand out.

AI Usage

We used AI for debugging and improving our testing outputs. For debugging this meant giving claude our code and error messages to see if it could find the solution. For testing this meant giving claude our testbench code and asking it to format the output in a simple visualization.